

AI Applications Lecture 2

Machine Learning Models and Parametric Functions

SUZUKI, Atsushi^a

^aI thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
4. What AI companies and users do mathematically? - Training and inference
5. Specifically, What is “training?” - Training algorithms
6. Summary and Future Outlook

Review of the previous lecture

In the previous lecture, we re-examined the diverse tasks that AI should solve from a unified perspective.

- The tasks we want computers to solve can be formulated as an **input-output relationship**.

Review of the previous lecture

In the previous lecture, we re-examined the diverse tasks that AI should solve from a unified perspective.

- The tasks we want computers to solve can be formulated as an **input-output relationship**.
- Real-world entities such as text, images, and audio can be converted into numerical data (vectors or tensors).

Review of the previous lecture

In the previous lecture, we re-examined the diverse tasks that AI should solve from a unified perspective.

- The tasks we want computers to solve can be formulated as an **input-output relationship**.
- Real-world entities such as text, images, and audio can be converted into numerical data (vectors or tensors).
- Thus, solving a task with AI boils down to a **function determination problem**: finding an appropriate **function** from an input space to an output space.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to perform the following tasks.

- Formulate a function determination problem as a parameter determination problem for a **parametric function**.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to perform the following tasks.

- Formulate a function determination problem as a parameter determination problem for a **parametric function**.
- Explain what parameters and functions constitute basic AI/machine learning models such as linear regression, logistic regression, decision trees, and k-NN.

Learning outcomes of this lecture

Through this lecture, students will aim to be able to perform the following tasks.

- Formulate a function determination problem as a parameter determination problem for a **parametric function**.
- Explain what parameters and functions constitute basic AI/machine learning models such as linear regression, logistic regression, decision trees, and k-NN.
- Formulate machine learning as determining parameters using data, and explain the difference between **training** and **inference**.

Policy of this lecture

The field of AI and machine learning is developing very rapidly, and specific models widely known today are likely to be outdated by the time you graduate.

Policy of this lecture

The field of AI and machine learning is developing very rapidly, and specific models widely known today are likely to be outdated by the time you graduate.

Therefore, this lecture emphasizes understanding more general and universal concepts that will be applicable in the future and will not become obsolete, rather than learning individual technologies.

Policy of this lecture

As a first step, we will learn about "parametric functions," a mathematical framework common to many machine learning models, including neural networks.

Policy of this lecture

As a first step, we will learn about "parametric functions," a mathematical framework common to many machine learning models, including neural networks.

Understanding this concept will allow you to:

- View various models from a unified perspective.
- Clearly distinguish between the two major phases of AI development: **training** and **inference**.

Policy of this lecture

As a first step, we will learn about "parametric functions," a mathematical framework common to many machine learning models, including neural networks.

Understanding this concept will allow you to:

- View various models from a unified perspective.
- Clearly distinguish between the two major phases of AI development: **training** and **inference**.

This distinction is an essential foundation for understanding neural network applications and practical issues such as licensing.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. Computer-friendly Representation of Functions? - Parametric Function

4. What AI companies and users do mathematically? - Training and inference

5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. Computer-friendly Representation of Functions? - Parametric Function

4. What AI companies and users do mathematically? - Training and inference

5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Preparation: mathematical notations

Here is a list of the basic mathematical notations used in this lecture.

- **Definition:**

- $(\text{LHS}) := (\text{RHS})$: The left-hand side is defined by the right-hand side.

Preparation: mathematical notations

Here is a list of the basic mathematical notations used in this lecture.

- **Definition:**

- $(\text{LHS}) := (\text{RHS})$: The left-hand side is defined by the right-hand side.

- **Set:** Denoted by uppercase calligraphic letters (e.g., $\mathcal{A}$).

- $x \in \mathcal{A}$: element x belongs to set $\mathcal{A}$.
- $\{a, b, c\}$: The set consisting of elements a, b, c .
- $\{x \in \mathcal{A} | P(x)\}$: The set of elements in $\mathcal{A}$ for which $P(x)$ is true.
- $\mathbb{R}$ (reals), $\mathbb{Z}$ (integers), with subscripts like $>_0$ (positive) or $\geq_0$ (non-negative).

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

Preparation: mathematical notations

- **Function:**

- $f : X \rightarrow Y$: f is a map from set X to set Y .
- $y = f(x)$: The output of f for input $x \in X$ is $y \in Y$.

- **Vector:** Denoted by bold italic lowercase letters (e.g., $\boldsymbol{v}$).

- A vector is a column of numbers.
- $\boldsymbol{v} \in \mathbb{R}^n$: an n -dimensional real vector.
- The i -th element is v_i . $\boldsymbol{v} = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}^\top$.

Preparation: mathematical notations

- **Matrix:** Denoted by bold italic uppercase letters (e.g., $\mathbf{A}$).
 - $\mathbf{A} \in \mathbb{R}^{m,n}$: an $m \times n$ real matrix.
 - The element in the i -th row, j -th column is $a_{i,j}$.
 - Transpose is denoted as $\mathbf{A}^\top$, where $(\mathbf{A}^\top)_{ij} = a_{ji}$.

Preparation: mathematical notations

- **Matrix:** Denoted by bold italic uppercase letters (e.g., $\mathbf{A}$).
 - $\mathbf{A} \in \mathbb{R}^{m,n}$: an $m \times n$ real matrix.
 - The element in the i -th row, j -th column is $a_{i,j}$.
 - Transpose is denoted as $\mathbf{A}^\top$, where $(\mathbf{A}^\top)_{ij} = a_{ji}$.
- **Tensor:**
 - In this lecture, a tensor is simply a multi-dimensional array.
 - Vector = 1st-order tensor, Matrix = 2nd-order tensor.
 - Higher-order tensors are denoted by $\underline{\mathbf{A}}$.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. Computer-friendly Representation of Functions? - Parametric Function

4. What AI companies and users do mathematically? - Training and inference

5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. Computer-friendly Representation of Functions? - Parametric Function

4. What AI companies and users do mathematically? - Training and inference

5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Question: Constructing an AI is a function determination problem. All a computer can directly handle is numerical data. How can we

Question: Constructing an AI is a function determination problem. All a computer can directly handle is numerical data. How can we represent candidate functions as numerical data?

Parametric functions

We have seen that solving a task can be formulated as a function determination problem.

Parametric functions

We have seen that solving a task can be formulated as a function determination problem.

Since we handle functions on a computer, the function must be specified by numerical data (e.g., a sequence of floating-point numbers).

Parametric functions

We have seen that solving a task can be formulated as a function determination problem.

Since we handle functions on a computer, the function must be specified by numerical data (e.g., a sequence of floating-point numbers).

Let's formalize this idea.

Definition (Parametric Function)

If there exists a set Θ (the **parameter space**), and for each element $\theta \in \Theta$, a function $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ is defined, then the family of functions $(f_\theta)_{\theta \in \Theta}$ is called a **parametric function**. Each θ is called a **parameter**.

Parametric functions

Given a parametric function, finding a function f is replaced by finding an appropriate parameter θ .

Parametric functions

Given a parametric function, finding a function f is replaced by finding an appropriate parameter θ .

Examples (other than neural networks):

Parametric functions

Given a parametric function, finding a function f is replaced by finding an appropriate parameter θ .

Examples (other than neural networks):

- **Linear/Logistic Regression:** Simple, foundational, and a special case of neural networks.

Parametric functions

Given a parametric function, finding a function f is replaced by finding an appropriate parameter θ .

Examples (other than neural networks):

- **Linear/Logistic Regression:** Simple, foundational, and a special case of neural networks.
- **Decision Trees:** A different kind of function not representable by typical NNs. Parameters are not a fixed-size vector. Intuitive and powerful.

Parametric functions

Given a parametric function, finding a function f is replaced by finding an appropriate parameter θ .

Examples (other than neural networks):

- **Linear/Logistic Regression:** Simple, foundational, and a special case of neural networks.
- **Decision Trees:** A different kind of function not representable by typical NNs. Parameters are not a fixed-size vector. Intuitive and powerful.
- **k-Nearest Neighbors (k-NN):** Another non-NN example. Also has variable-size parameters. A good example on which to see the training and inference steps later, since its training algorithm is simple.

Example 1: linear regression

Considers mapping a d_{in} -dimensional vector $x \in \mathbb{R}^{d_{\text{in}}}$ to a 1-dimensional real number $y \in \mathbb{R}$.

Example 1: linear regression

Considers mapping a d_{in} -dimensional vector $x \in \mathbb{R}^{d_{\text{in}}}$ to a 1-dimensional real number $y \in \mathbb{R}$.

- **Parameters:** $\theta := (w, b)$, where $w \in \mathbb{R}^{d_{\text{in}}}$ is a weight vector and $b \in \mathbb{R}$ is a bias.

Example 1: linear regression

Considers mapping a d_{in} -dimensional vector $\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}}$ to a 1-dimensional real number $y \in \mathbb{R}$.

- **Parameters:** $\boldsymbol{\theta} := (\boldsymbol{w}, b)$, where $\boldsymbol{w} \in \mathbb{R}^{d_{\text{in}}}$ is a weight vector and $b \in \mathbb{R}$ is a bias.
- **Function:** $f_{\boldsymbol{w}, b}(\boldsymbol{x}) := \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{i=1}^{d_{\text{in}}} w_i x_i + b$.

Example 1: linear regression

Considers mapping a d_{in} -dimensional vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to a 1-dimensional real number $y \in \mathbb{R}$.

- **Parameters:** $\theta := (\mathbf{w}, b)$, where $\mathbf{w} \in \mathbb{R}^{d_{\text{in}}}$ is a weight vector and $b \in \mathbb{R}$ is a bias.
- **Function:** $f_{\mathbf{w},b}(\mathbf{x}) := \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^{d_{\text{in}}} w_i x_i + b$.

Example

For $d_{\text{in}} = 2$, parameters $\mathbf{w} = \begin{bmatrix} 1.0 \\ -2.0 \end{bmatrix}$, $b = 0.5$, and input $\mathbf{x} = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$, the output is:

$$\begin{aligned} y = f_{(\mathbf{w},b)}(\mathbf{x}) &= (1.0 \times 3.0) + (-2.0 \times 4.0) + 0.5 \\ &= 3.0 - 8.0 + 0.5 = -4.5 \end{aligned}$$

Example 1: linear regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, calculate the output for the same input as in the previous example, $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$.

Example 1: linear regression

Exercise

For parameters $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$, calculate the output for the same input as in the previous example, $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$.

Answer:

$$\begin{aligned} y = f_{(w,b)}(x) &= (-0.5 \times 3.0) + (3.0 \times 4.0) + (-1.0) \\ &= -1.5 + 12.0 - 1.0 = 9.5 \end{aligned}$$

(This shows that the output changes with different parameters for the same input.)

Example 2: logistic regression

The input is $x \in \mathbb{R}^{d_{\text{in}}}$, but the output is a probability value $y \in [0, 1]$.

Example 2: logistic regression

The input is $x \in \mathbb{R}^{d_{\text{in}}}$, but the output is a probability value $y \in [0, 1]$.

- **Parameters:** Same as linear regression, $\theta := (w, b)$.

Example 2: logistic regression

The input is $x \in \mathbb{R}^{d_{\text{in}}}$, but the output is a probability value $y \in [0, 1]$.

- **Parameters:** Same as linear regression, $\theta := (w, b)$.
- **Function:** $f_{w,b}(x) := \sigma(w^\top x + b)$.

Example 2: logistic regression

The input is $x \in \mathbb{R}^{d_{\text{in}}}$, but the output is a probability value $y \in [0, 1]$.

- **Parameters:** Same as linear regression, $\theta := (w, b)$.
- **Function:** $f_{w,b}(x) := \sigma(w^\top x + b)$.
- Here, $\sigma(z) := \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

Example 2: logistic regression

Example

Using the same parameters and input as the first linear regression example

($w = \begin{bmatrix} 1.0 \\ -2.0 \end{bmatrix}$, $b = 0.5$, $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$), we first calculate the linear part

$$z = w^\top x + b = -4.5.$$

The final output is:

$$\begin{aligned} y = \sigma(-4.5) &= \frac{1}{1 + e^{4.5}} \\ &\approx \frac{1}{1 + 90.017} \approx 0.011 \end{aligned}$$

Example 2: logistic regression

Exercise

For the same input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$, calculate the output using the parameters from the linear regression exercise, $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$.

Example 2: logistic regression

Exercise

For the same input $x = \begin{bmatrix} 3.0 \\ 4.0 \end{bmatrix}$, calculate the output using the parameters from the linear regression exercise, $w = \begin{bmatrix} -0.5 \\ 3.0 \end{bmatrix}$, $b = -1.0$.

Answer: First, the linear part is $z = w^\top x + b = 9.5$. The final output is:

$$\begin{aligned} y = \sigma(9.5) &= \frac{1}{1 + e^{-9.5}} \\ &\approx \frac{1}{1 + 0.0000748} \approx 0.999925 \end{aligned}$$

Example 3: decision tree / regression tree

Decision trees are examples where parameters are not a fixed-length vector. For an input $x \in \mathbb{R}^{d_{\text{in}}}$, the output is determined by traversing a tree structure.

Example 3: decision tree / regression tree

Decision trees are examples where parameters are not a fixed-length vector. For an input $x \in \mathbb{R}^{d_{\text{in}}}$, the output is determined by traversing a tree structure.

Mathematical Formulation:

- A binary tree structure with nodes indexed by integers.
- **Parameters θ define:**
 - The set of nodes in the tree ($\mathcal{V}$).
 - For each **internal node**: which input feature to check (j_i) and a threshold (t_i).
 - For each **leaf node**: the output value (c_l).
- **Function $f_{\theta}(x)$:**
 - Start at the root node.
 - At each internal node, compare x_{j_i} with t_i to decide whether to go left or right.
 - When a leaf node is reached, return its value c_l .

Example 3: regression tree

Example (Regression Tree)

Let $d_{\text{in}} = 2$. Consider a tree defined by parameters θ_1 :

- Internal nodes:
 - Node 1: split on $x_1 < 2.5$
 - Node 3: split on $x_2 < 0.0$
- Leaf nodes:
 - Node 2: value = 10.0
 - Node 6: value = -5.0
 - Node 7: value = 8.0

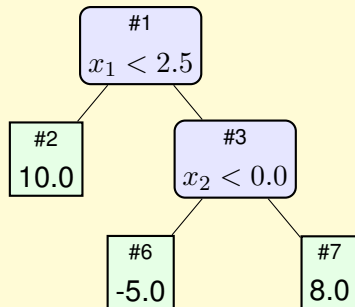


Figure 1: Parameters θ_1

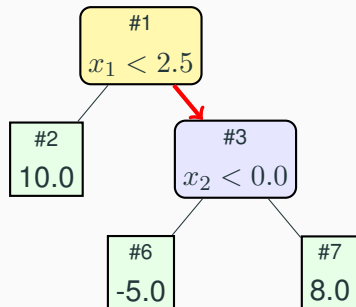
Calculate the output for input $x = \begin{bmatrix} 3.0 \\ -1.0 \end{bmatrix}$.

Example 3: regression tree

Input: $x = \begin{bmatrix} 3.0 \\ -1.0 \end{bmatrix}$

Step 1: Evaluate at root node 1.

- The rule is: $x_1 < 2.5$.
- Our input has $x_1 = 3.0$.
- Since $3.0 \geq 2.5$, the condition is false.
- We proceed to the **right** child, node 3.

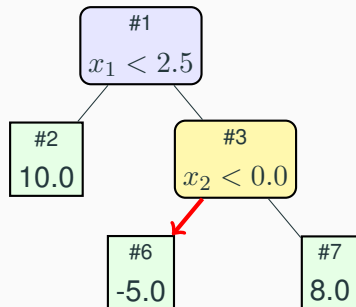


Example 3: regression tree

Input: $x = \begin{bmatrix} 3.0 \\ -1.0 \end{bmatrix}$

Step 2: Evaluate at node 3.

- The rule is: $x_2 < 0.0$.
- Our input has $x_2 = -1.0$.
- Since $-1.0 < 0.0$, the condition is true.
- We proceed to the **left** child, node 6.



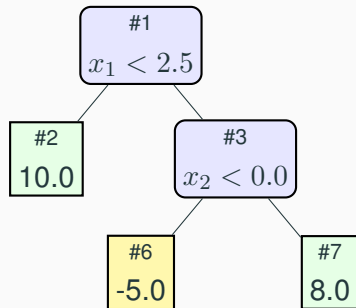
Example 3: regression tree

Input: $x = \begin{bmatrix} 3.0 \\ -1.0 \end{bmatrix}$

Step 3: Arrive at leaf node 6.

- Node 6 is a leaf node.
- The value associated with this leaf is $c_6 = -5.0$.
- This is our final output.

Therefore, the output is $y = f_{\theta_1}(x) = -5.0$.



Example 3: regression tree

Exercise

Consider a different tree defined by parameters θ_2 . Calculate the output for the same input $x = \begin{bmatrix} 3.0 \\ -1.0 \end{bmatrix}$.

- Internal nodes:
 - Node 1: $x_2 < 5.0$
 - Node 2: $x_1 < 1.0$
- Leaf nodes:
 - Node 3: 20.0
 - Node 4: 1.5
 - Node 5: 3.0

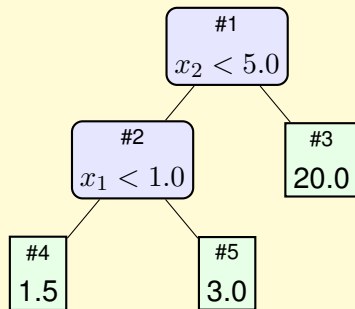


Figure 2: Parameters θ_2

Example 3: regression tree

Answer:

1. At node 1 (root): $x_2 = -1.0 < 5.0$ (true) $\rightarrow$ go left to node 2.
2. At node 2: $x_1 = 3.0 \geq 1.0$ (false) $\rightarrow$ go right to node 5.
3. Node 5 is a leaf node. Its value is $c_5 = 3.0$.

Therefore, the output is $y = f_{\theta_2}(x) = 3.0$.

Example 4: k-nearest neighbors (k-NN)

k-NN is another model where the parameters are of variable length.

Example 4: k-nearest neighbors (k-NN)

k-NN is another model where the parameters are of variable length.

Mathematical Formulation: ($k \in \mathbb{N}$ is fixed in advance)

- **Parameters:** Input-output pairs $\theta := ((x_1, y_1), \dots, (x_N, y_N))$.

Example 4: k-nearest neighbors (k-NN)

k-NN is another model where the parameters are of variable length.

Mathematical Formulation: ($k \in \mathbb{N}$ is fixed in advance)

- **Parameters:** Input-output pairs $\theta := ((x_1, y_1), \dots, (x_N, y_N))$.
- **Function:** For a new input x_{new} :
 1. Calculate the distance from x_{new} to every x_i in the dataset.
 2. Find the k data points (x_i, y_i) with the smallest distances.
 3. Aggregate their outputs y_i to get the final result. (e.g., mean for regression, majority vote for classification).

Example 4: k-NN regression

Example (Regression)

Let $k = 3$. The parameters θ_1 are the dataset $\mathcal{D}_1$:

$$\mathcal{D}_1 = \left(\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, 5 \right), \left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}, 10 \right), \left(\begin{bmatrix} 3 \\ 0 \end{bmatrix}, 20 \right), \left(\begin{bmatrix} 4 \\ 3 \end{bmatrix}, 25 \right) \right)$$

Calculate the output for a new input $\mathbf{x}_{new} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

Example 4: k-NN regression

Input: $\mathbf{x}_{new} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. **Data:** $\mathcal{D}_1$.

Step 1: Calculate squared Euclidean distance to each data point.

- $d(\mathbf{x}_{new}, \mathbf{x}_1)^2 = (2 - 0)^2 + (1 - 0)^2 = 4 + 1 = 5$
- $d(\mathbf{x}_{new}, \mathbf{x}_2)^2 = (2 - 1)^2 + (1 - 2)^2 = 1 + 1 = 2$
- $d(\mathbf{x}_{new}, \mathbf{x}_3)^2 = (2 - 3)^2 + (1 - 0)^2 = 1 + 1 = 2$
- $d(\mathbf{x}_{new}, \mathbf{x}_4)^2 = (2 - 4)^2 + (1 - 3)^2 = 4 + 4 = 8$

Example 4: k-NN regression

Step 2: Find the top $k = 3$ nearest neighbors.

- The distances are: 5, 2, 2, 8.
- The smallest three distances correspond to data points x_2 (dist 2), x_3 (dist 2), and x_1 (dist 5).
- Their corresponding outputs are $y_2 = 10$, $y_3 = 20$, $y_1 = 5$.

Example 4: k-NN regression

Step 2: Find the top $k = 3$ nearest neighbors.

- The distances are: 5, 2, 2, 8.
- The smallest three distances correspond to data points x_2 (dist 2), x_3 (dist 2), and x_1 (dist 5).
- Their corresponding outputs are $y_2 = 10$, $y_3 = 20$, $y_1 = 5$.

Step 3: Aggregate the outputs (mean for regression).

- We calculate the average of the neighbors' output values.

$$y = \frac{10 + 20 + 5}{3} = \frac{35}{3} \approx 11.67$$

The final output is $y \approx 11.67$.

Example 4: k-NN regression

Exercise

Let $k = 2$ and assume the parameters θ_2 consist of the following five data points.

$$\mathcal{D}_2 = ((\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1), (\begin{bmatrix} 2 \\ 3 \end{bmatrix}, 4), (\begin{bmatrix} -1 \\ 2 \end{bmatrix}, 6), (\begin{bmatrix} 0 \\ -1 \end{bmatrix}, 8), (\begin{bmatrix} 4 \\ 0 \end{bmatrix}, 12))$$

Calculate the output for the same new input as in the previous example,

$$\mathbf{x}_{new} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Example 4: k-NN regression

Answer:

- Squared distances are: 1, 4, 10, 8, 5.
- The two nearest neighbors are $(\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1)$ and $(\begin{bmatrix} 2 \\ 3 \end{bmatrix}, 4)$.
- Their outputs are $\{1, 4\}$.
- The mean is $y = \frac{1+4}{2} = 2.5$.

Example 4: k-NN regression

Remark

Models like k-NN, which store the training data itself or have a non-fixed number of parameters, are often called **non-parametric** models.

Example 4: k-NN regression

Remark

Models like k-NN, which store the training data itself or have a non-fixed number of parameters, are often called **non-parametric** models.

This is historical terminology and slightly conflicts with this lecture's definition of a "parametric function."

Example 4: k-NN regression

Remark

Models like k-NN, which store the training data itself or have a non-fixed number of parameters, are often called **non-parametric** models.

This is historical terminology and slightly conflicts with this lecture's definition of a "parametric function."

In this lecture, we consider non-parametric models as a type of "parametric function with a parameter whose structure depends on the data length."

Representation of function - answer

Question: Constructing an AI is a function determination problem. All a computer can directly handle is numerical data. How can we represent candidate functions as numerical data?

Answer:

Representation of function - answer

Question: Constructing an AI is a function determination problem. All a computer can directly handle is numerical data. How can we represent candidate functions as numerical data?

Answer: Associate each function to a numerical **parameter**.

Outline

1. Introduction

2. Preparation: Mathematical Notations

3. Computer-friendly Representation of Functions? - Parametric Function

4. What AI companies and users do mathematically? - Training and inference

5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
- 4. What AI companies and users do mathematically? - Training and inference**
5. Specifically, What is “training?” - Training algorithms
6. Summary and Future Outlook

Question: How do AI tech companies and we, users, use parametric functions as AI?

Scheme of training and inference

Viewing models as parametric functions, $(f_{\theta})_{\theta \in \Theta}$, separates the problem-solving process into **training** phase and **inference** phase.

- **Training:**

Scheme of training and inference

Viewing models as parametric functions, $(f_{\theta})_{\theta \in \Theta}$, separates the problem-solving process into **training** phase and **inference** phase.

- **Training:** The process of finding the "optimal" parameters $\theta^* \in \Theta$ that can best solve the task, using given **training data** $\mathcal{D}$. This is often called **fitting**.

$$\theta^* = \mathfrak{A}((f_{\theta}), \mathcal{D})$$

- **Inference:**

Scheme of training and inference

Viewing models as parametric functions, $(f_{\theta})_{\theta \in \Theta}$, separates the problem-solving process into **training** phase and **inference** phase.

- **Training:** The process of finding the "optimal" parameters $\theta^* \in \Theta$ that can best solve the task, using given **training data** $\mathcal{D}$. This is often called **fitting**.

$$\theta^* = \mathfrak{A}((f_{\theta}), \mathcal{D})$$

- **Inference:** The process of fixing the parameters θ^* and using the specific function f_{θ^*} to calculate the output y for a new, unknown input x . This is also called **prediction**.

$$y = f_{\theta^*}(x)$$

Most cases,

Scheme of training and inference

Viewing models as parametric functions, $(f_{\theta})_{\theta \in \Theta}$, separates the problem-solving process into **training** phase and **inference** phase.

- **Training:** The process of finding the "optimal" parameters $\theta^* \in \Theta$ that can best solve the task, using given **training data** $\mathcal{D}$. This is often called **fitting**.

$$\theta^* = \mathfrak{A}((f_{\theta}), \mathcal{D})$$

- **Inference:** The process of fixing the parameters θ^* and using the specific function f_{θ^*} to calculate the output y for a new, unknown input x . This is also called **prediction**.

$$y = f_{\theta^*}(x)$$

Most cases, an AI company trains AI, and users do inference by the AI.

Scheme of training and inference

Training
(by AI company)

Parametric
Function $f_{(\cdot)}$

Training Data $\mathcal{D}$

Learning Algorithm $\mathfrak{A}$

Optimal parameter θ^*



Scheme of training and inference

Training

(by AI company)

Parametric
Function $f_{(\cdot)}$



Training Data $\mathcal{D}$



Learning Algorithm $\mathfrak{A}$



Optimal parameter θ^*



New Input x



Function f_{θ^*} with the
optimal parameter θ^*



Predicted Output y

Inference

(by user)

Scheme of training and inference

Remark (Why Emphasize the Difference Between Training and Inference?)

The distinction is essential for understanding practical issues when using neural network models, such as publication and licensing.

Scheme of training and inference

Remark (Why Emphasize the Difference Between Training and Inference?)

The distinction is essential for understanding practical issues when using neural network models, such as publication and licensing.

- For practical neural networks, the parameters for inference (θ^*) are often public, while the training algorithm ($\mathfrak{A}$) and data ($\mathcal{D}$) are not.

Scheme of training and inference

Remark (Why Emphasize the Difference Between Training and Inference?)

The distinction is essential for understanding practical issues when using neural network models, such as publication and licensing.

- For practical neural networks, the parameters for inference (θ^*) are often public, while the training algorithm ($\mathfrak{A}$) and data ($\mathcal{D}$) are not.
- The license for the inference part and the training part may not be the same.

Scheme of training and inference

Remark (Why Emphasize the Difference Between Training and Inference?)

The distinction is essential for understanding practical issues when using neural network models, such as publication and licensing.

- For practical neural networks, the parameters for inference (θ^*) are often public, while the training algorithm ($\mathfrak{A}$) and data ($\mathcal{D}$) are not.
- The license for the inference part and the training part may not be the same.
- Handling neural network models without understanding their licenses carries legal risks.

Training and inference - answer

Question: How do AI tech companies and we, users, use parametric functions as AI?

Answer:

Training and inference - answer

Question: How do AI tech companies and we, users, use parametric functions as AI?

Answer:

- An AI company

Training and inference - answer

Question: How do AI tech companies and we, users, use parametric functions as AI?

Answer:

- An AI company **trains** an AI, i.e., finds an appropriate parameter of a parametric function.
- A user

Training and inference - answer

Question: How do AI tech companies and we, users, use parametric functions as AI?

Answer:

- An AI company **trains** an AI, i.e., finds an appropriate parameter of a parametric function.
- A user lets the AI do **infer**, i.e., applies the parametric function with the parameter determined by the AI company.

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
- 4. What AI companies and users do mathematically? - Training and inference**
5. Specifically, What is “training?” - Training algorithms
6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
4. What AI companies and users do mathematically? - Training and inference
- 5. Specifically, What is “training?” - Training algorithms**
6. Summary and Future Outlook

Question: What is “training,” specifically?

Specific examples of training algorithms

Inference is simply the computation $y = f_{\theta^*}(x)$. All the calculation examples we have seen so far correspond to this inference phase.

Specific examples of training algorithms

Inference is simply the computation $y = f_{\theta^*}(x)$. All the calculation examples we have seen so far correspond to this inference phase.

The training phase, finding the "optimal" parameters θ^* from data $\mathcal{D}$, differs greatly depending on the type of parametric function.

Specific examples of training algorithms

Inference is simply the computation $\mathbf{y} = f_{\theta^*}(\mathbf{x})$. All the calculation examples we have seen so far correspond to this inference phase.

The training phase, finding the "optimal" parameters θ^* from data $\mathcal{D}$, differs greatly depending on the type of parametric function.

The goal here is NOT to memorize algorithms, but

Specific examples of training algorithms

Inference is simply the computation $\mathbf{y} = f_{\theta^*}(\mathbf{x})$. All the calculation examples we have seen so far correspond to this inference phase.

The training phase, finding the "optimal" parameters θ^* from data $\mathcal{D}$, differs greatly depending on the type of parametric function.

The goal here is NOT to memorize algorithms, but to recognize the advantage of neural networks: Different parametric functions require different training algorithms, but, in contrast, the uniform learning scheme can be applied to all neural network variants!

Example 1: training of k-NN

The learning algorithm for k-NN is one of the simplest imaginable.

Example 1: training of k-NN

The learning algorithm for k-NN is one of the simplest imaginable.

The learning algorithm $\mathfrak{A}$ simply stores the given training data

$\mathcal{D} = ((\boldsymbol{x}_1, y_1), \dots, (\boldsymbol{x}_N, y_N))$ as the parameters θ .

Example 1: training of k-NN

The learning algorithm for k-NN is one of the simplest imaginable.

The learning algorithm $\mathfrak{A}$ simply stores the given training data $\mathcal{D} = ((\boldsymbol{x}_1, y_1), \dots, (\boldsymbol{x}_N, y_N))$ as the parameters θ .

Training is completed just by "memorizing the data."

Example 1: training of k-NN

The learning algorithm for k-NN is one of the simplest imaginable.

The learning algorithm $\mathcal{A}$ simply stores the given training data $\mathcal{D} = ((\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N))$ as the parameters θ .

Training is completed just by "memorizing the data."

Models like this, where most of the computation is deferred until inference time, are sometimes called **lazy learning** models.

Example 2: training of linear regression

For models with fixed-length numerical parameters like linear regression, training is a more active process.

Example 2: training of linear regression

For models with fixed-length numerical parameters like linear regression, training is a more active process.

Training is often formulated as an **optimization problem**: "minimizing a function that measures the 'badness' of the model's predictions on the training data."

Example 2: training of linear regression

For models with fixed-length numerical parameters like linear regression, training is a more active process.

Training is often formulated as an **optimization problem**: "minimizing a function that measures the 'badness' of the model's predictions on the training data."

Definition (Loss Function)

A function $L : \Theta \rightarrow \mathbb{R}_{\geq 0}$ that takes parameters θ and outputs a non-negative real value representing how "bad" the model is for the data is called a **cost function**, **loss function**, or **objective function**. The goal is to find:

$$\theta^* := \arg \min_{\theta \in \Theta} L(\theta)$$

Least squares method

For linear regression, a widely used loss function is the **Sum of Squared Residuals (SSR)**.

Least squares method

For linear regression, a widely used loss function is the **Sum of Squared Residuals (SSR)**.

The method of minimizing this loss is called the **least squares method**.

Least squares method

For linear regression, a widely used loss function is the **Sum of Squared Residuals (SSR)**.

The method of minimizing this loss is called the **least squares method**.

Definition (Sum of Squared Residuals)

Given training data $\mathcal{D} = ((\mathbf{x}_i, y_i))_{i=1}^N$, the SSR is the sum of the squares of the differences (the **residuals**) between the actual outputs y_i and the model's predicted values $f(\mathbf{x}_i)$.

$$\begin{aligned} L_{\text{SSR}}(\mathbf{w}, b) &:= \sum_{i=1}^N (y_i - f_{(\mathbf{w}, b)}(\mathbf{x}_i))^2 \\ &= \sum_{i=1}^N (y_i - (\mathbf{w}^\top \mathbf{x}_i + b))^2 \end{aligned}$$

Least squares solution

The least squares solution can be found analytically using linear algebra.

Least squares solution

The least squares solution can be found analytically using linear algebra.

Step 1: Augment vectors and matrices.

- Append a 1 to each input vector: $\tilde{\mathbf{x}}_i := \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix}$.
- Combine weights and bias: $\tilde{\mathbf{w}} := \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$.
- Now, the prediction is a simple dot product: $f(\mathbf{x}_i) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i$.

Least squares solution

Step 2: Represent data in matrix form.

- Design Matrix $\mathbf{X}$: Each row is an augmented input vector $\tilde{\mathbf{x}}_i^\top$.
- Target Vector $\mathbf{y}$: A column vector of all true outputs y_i .

$$\mathbf{X} := \begin{bmatrix} \tilde{\mathbf{x}}_1^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{bmatrix}, \quad \mathbf{y} := \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}$$

Least squares solution

With the matrix formulation, the vector of all predictions $\hat{\mathbf{y}}$ is simply $\hat{\mathbf{y}} = \mathbf{X}\tilde{\mathbf{w}}$.

Least squares solution

With the matrix formulation, the vector of all predictions $\hat{\mathbf{y}}$ is simply $\hat{\mathbf{y}} = \mathbf{X}\tilde{\mathbf{w}}$.

The loss function SSR becomes the squared L2-norm of the difference vector:

$$L_{\text{SSR}}(\tilde{\mathbf{w}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 = \|\mathbf{y} - \mathbf{X}\tilde{\mathbf{w}}\|_2^2$$

Least squares solution

With the matrix formulation, the vector of all predictions $\hat{\mathbf{y}}$ is simply $\hat{\mathbf{y}} = \mathbf{X}\tilde{\mathbf{w}}$.

The loss function SSR becomes the squared L2-norm of the difference vector:

$$L_{\text{SSR}}(\tilde{\mathbf{w}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2 = \|\mathbf{y} - \mathbf{X}\tilde{\mathbf{w}}\|_2^2$$

The minimizer of this loss function is the solution to the following equation:

Definition (Normal Equation)

The following equation for $\tilde{\mathbf{w}}$ is called the **normal equation**.

$$\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\tilde{\mathbf{w}}) = \mathbf{0}. \tag{1}$$

Least squares solution

Theorem (Least Squares Solution)

A vector $\tilde{w}$ minimizes the Sum of Squared Residuals (SSR) if and only if it is a solution to the normal equation.

Least squares solution

Theorem (Least Squares Solution)

A vector $\tilde{w}$ minimizes the Sum of Squared Residuals (SSR) if and only if it is a solution to the normal equation.

In particular, if the matrix $\mathbf{X}^\top \mathbf{X}$ is invertible, the normal equation has a unique solution given by:

$$\tilde{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (2)$$

Least squares solution

Theorem (Least Squares Solution)

A vector $\tilde{w}$ minimizes the Sum of Squared Residuals (SSR) if and only if it is a solution to the normal equation.

In particular, if the matrix $\mathbf{X}^\top \mathbf{X}$ is invertible, the normal equation has a unique solution given by:

$$\tilde{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (2)$$

Geometric Interpretation: The least squares method finds the **orthogonal projection** of the target vector $\mathbf{y}$ onto the subspace spanned by the columns of the design matrix $\mathbf{X}$. The normal equation $\mathbf{X}^\top (\mathbf{y} - \mathbf{X} \tilde{w}) = \mathbf{0}$ expresses the condition that the error vector $(\mathbf{y} - \mathbf{X} \tilde{w})$ must be orthogonal to that subspace.

Least squares solution

Example

Let's find the optimal parameters for the following training data $\mathcal{D}_A$:

$$\mathcal{D}_A = ((\mathbf{x}_1, y_1) = (\begin{bmatrix} 3 \\ 4 \end{bmatrix}, -4.5), (\mathbf{x}_2, y_2) = (\begin{bmatrix} 1 \\ 0 \end{bmatrix}, 1.5), (\mathbf{x}_3, y_3) = (\begin{bmatrix} 0 \\ 1 \end{bmatrix}, -1.5))$$

Least squares solution

Step 1: Construct the design matrix X and target vector y .

- The input vectors are $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$, $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$.
- The augmented input vectors (add a 1) are $\begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$.
- The outputs are $-4.5, 1.5, -1.5$.

Least squares solution

Step 1: Construct the design matrix X and target vector y .

- The input vectors are $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$, $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$.
- The augmented input vectors (add a 1) are $\begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$.
- The outputs are $-4.5, 1.5, -1.5$.

$$X = \begin{bmatrix} 3 & 4 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} -4.5 \\ 1.5 \\ -1.5 \end{bmatrix}$$

Least squares solution

Step 2: Calculate $X^T X$ and $X^T y$.

$$\begin{aligned} X^T X &= \begin{bmatrix} 3 & 1 & 0 \\ 4 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 4 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 10 & 12 & 4 \\ 12 & 17 & 5 \\ 4 & 5 & 3 \end{bmatrix} \end{aligned}$$

Least squares solution

Step 2: Calculate $X^T X$ and $X^T y$. (Continued)

$$\begin{aligned} X^T y &= \begin{bmatrix} 3 & 1 & 0 \\ 4 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} -4.5 \\ 1.5 \\ -1.5 \end{bmatrix} \\ &= \begin{bmatrix} -13.5 + 1.5 \\ -18.0 - 1.5 \\ -4.5 + 1.5 - 1.5 \end{bmatrix} = \begin{bmatrix} -12.0 \\ -19.5 \\ -4.5 \end{bmatrix} \end{aligned}$$

Least squares solution

Step 3: Solve the normal equation $(X^\top X)\tilde{w} = X^\top y$.

$$\begin{bmatrix} 10 & 12 & 4 \\ 12 & 17 & 5 \\ 4 & 5 & 3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} = \begin{bmatrix} -12.0 \\ -19.5 \\ -4.5 \end{bmatrix}$$

Least squares solution

Step 3: Solve the normal equation $(X^\top X)\tilde{w} = X^\top y$.

$$\begin{bmatrix} 10 & 12 & 4 \\ 12 & 17 & 5 \\ 4 & 5 & 3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} = \begin{bmatrix} -12.0 \\ -19.5 \\ -4.5 \end{bmatrix}$$

Solving this system of linear equations (e.g., by finding the inverse of the matrix on the left), we get the solution:

$$\tilde{w}^* = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} = \begin{bmatrix} 1.0 \\ -2.0 \\ 0.5 \end{bmatrix}$$

These are exactly the parameters we used in the first inference example!

Diversity of training algorithms and neural networks

As we have seen, Training Algorithms can be completely different:

- **k-NN:** Just store the data ("lazy learning").
- **Linear Regression:** Solve a system of linear equations (the normal equation).
- **Decision Trees:** Use a greedy algorithm to find the best splits (not detailed here).

Diversity of training algorithms and neural networks

As we have seen, Training Algorithms can be completely different:

- **k-NN:** Just store the data ("lazy learning").
- **Linear Regression:** Solve a system of linear equations (the normal equation).
- **Decision Trees:** Use a greedy algorithm to find the best splits (not detailed here).

In contrast, one of the greatest strengths of **neural networks** is the **uniformity of their learning algorithm**.

Diversity of training algorithms and neural networks

As we have seen, Training Algorithms can be completely different:

- **k-NN:** Just store the data ("lazy learning").
- **Linear Regression:** Solve a system of linear equations (the normal equation).
- **Decision Trees:** Use a greedy algorithm to find the best splits (not detailed here).

In contrast, one of the greatest strengths of **neural networks** is the **uniformity of their learning algorithm**.

Most neural networks can be trained under a common framework:

1. Calculate the gradient of the loss function using **backpropagation**.
2. Minimize the loss using **stochastic gradient descent (SGD)** or its variants.

Training algorithms - answer

Question: What is “training,” specifically?

Answer:

Training algorithms - answer

Question: What is “training,” specifically?

Answer: The training algorithm depends on what parametric function we use.
However,

Training algorithms - answer

Question: What is “training,” specifically?

Answer: The training algorithm depends on what parametric function we use. However, if we use a neural network, we can always train it using backpropagation and stochastic gradient descent.

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
4. What AI companies and users do mathematically? - Training and inference
- 5. Specifically, What is “training?” - Training algorithms**
6. Summary and Future Outlook

Outline

1. Introduction
2. Preparation: Mathematical Notations
3. Computer-friendly Representation of Functions? - Parametric Function
4. What AI companies and users do mathematically? - Training and inference
5. Specifically, What is “training?” - Training algorithms

6. Summary and Future Outlook

Today's summary

- A task can be reduced to a parameter determination problem for a **parametric function**.

Today's summary

- A task can be reduced to a parameter determination problem for a **parametric function**.
- Various AI models like linear regression, decision trees, and k-NN can be understood uniformly as parametric functions.

Today's summary

- A task can be reduced to a parameter determination problem for a **parametric function**.
- Various AI models like linear regression, decision trees, and k-NN can be understood uniformly as parametric functions.
- Machine learning is separated into a **training** phase (finding optimal parameters from data) and an **inference** phase (using those parameters to make predictions).

Next time

This time, we viewed various machine learning models within the unified framework of parametric functions. This is an important foundation for understanding neural networks.

Next time

This time, we viewed various machine learning models within the unified framework of parametric functions. This is an important foundation for understanding neural networks.

Next time, we will strictly define what a **neural network** is—a particularly expressive type of parametric function. We will use the mathematical tool of a **Directed Acyclic Graph (DAG)** to do this.

Next time

This time, we viewed various machine learning models within the unified framework of parametric functions. This is an important foundation for understanding neural networks.

Next time, we will strictly define what a **neural network** is—a particularly expressive type of parametric function. We will use the mathematical tool of a **Directed Acyclic Graph (DAG)** to do this.

Then, we will learn about the distinction between "architecture" and "checkpoints," concepts that are extremely important in the practical application of modern AI.

Appendix: differentiation by a vector i

The gradient of a scalar function $f(x)$ with respect to a vector x is defined as a vector of partial derivatives.

$$\nabla_x f(x) = \frac{\partial f(x)}{\partial x} := \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} \quad \cdots \quad \frac{\partial f}{\partial x_n} \right]^\top$$

1. Derivative of a Linear Term:

$$\frac{\partial(a^\top x)}{\partial x} = a$$

2. Derivative of a Quadratic Form: For a symmetric matrix A :

$$\frac{\partial(x^\top Ax)}{\partial x} = 2Ax \quad (\text{if } A = A^\top)$$

Appendix: differentiation by a vector ii

Recall the loss function:

$$L(\tilde{\mathbf{w}}) = \mathbf{y}^\top \mathbf{y} - 2\tilde{\mathbf{w}}^\top \mathbf{X}^\top \mathbf{y} + \tilde{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X}) \tilde{\mathbf{w}}$$

Differentiating each term w.r.t. $\tilde{\mathbf{w}}$:

- 1st term ($\mathbf{y}^\top \mathbf{y}$): Constant, derivative is 0.
- 2nd term ($-2\tilde{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{y})$): Linear term, derivative is $-2\mathbf{X}^\top \mathbf{y}$.
- 3rd term ($\tilde{\mathbf{w}}^\top (\mathbf{X}^\top \mathbf{X}) \tilde{\mathbf{w}}$): Quadratic form with a symmetric matrix ($\mathbf{X}^\top \mathbf{X}$), derivative is $2(\mathbf{X}^\top \mathbf{X}) \tilde{\mathbf{w}}$.

Summing them up gives the gradient:

$$\nabla_{\tilde{\mathbf{w}}} L = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X} \tilde{\mathbf{w}}$$